

MODELAGEM E DESENVOLVIMENTO ORIENTADO A OBJETOS

Projeto em Grupo - Academia: Planos e Frequência

Integrantes do Grupo:

Daniel Ponte Arruda (RA: 3024103499),
Eduardo da Silva Domingues (RA: 3024103409),
Fernando Alves Sobrinho (RA: 3024103405),
Isabelle Skirda Trindade (RA: 3024104316),
João Vitor Valentini (RA: 3024105232),
Kayky Pereira Távora (RA: 3024104399)
Lucas Toneli dos Santos (RA: 3024104377).

1 INTRODUÇÃO

O presente trabalho dá continuidade ao projeto em grupo desenvolvido na disciplina de Modelagem e Desenvolvimento Orientado a Objetos, aplicando os conceitos de análise e projeto orientados a objetos estudados em aula ao sistema escolhido pelo grupo: o controle de planos e frequência de uma academia.

O sistema tem como objetivo auxiliar uma academia no controle de seus alunos, planos e frequência. Ele deve permitir o cadastro e o gerenciamento dos alunos, associar cada aluno a um plano e registrar seus check-ins. Uma das principais regras do domínio é que um aluno não pode realizar check-in caso o seu plano esteja vencido ou inativo.

Para esta etapa da modelagem, foram selecionadas as classes Aluno, Plano e CheckIn como principais classes do domínio, sobre as quais se apoiam o diagrama de classes, o diagrama de objetos, a mensagem trocada entre objetos e a identificação dos quatro pilares da Orientação a Objetos, apresentados nas seções a seguir.

1.1 Aderência aos requisitos solicitados

O quadro a seguir relaciona cada um dos itens obrigatórios definidos pelo professor à seção deste documento em que ele é atendido.

Requisito solicitado	Onde é atendido
Diagrama de classes	Seção 3
Diagrama de objetos	Seção 4
3 classes do domínio, com 2–3 atributos e 1–2 operações cada	Seção 2
Mensagem entre dois objetos do sistema	Seção 5
Abstração, Encapsulamento, Herança e Polimorfismo no modelo	Seção 6

2 CLASSES DO DOMÍNIO

Foram selecionadas três classes centrais do domínio da academia. Para cada uma são apresentados os atributos, as operações e uma breve descrição de sua responsabilidade no sistema.

2.1 Classe Aluno

Representa uma pessoa matriculada na academia.

Atributo	Tipo
nome	String
cpf	String
plano	Plano

Operação	Retorno
realizarCheckIn(checkIn: CheckIn)	boolean
consultarPlano()	Plano

2.2 Classe Plano

Representa o plano contratado pelo aluno e o seu estado atual.

Atributo	Tipo
tipo	String
valor	double
status	String

Operação	Retorno
estaAtivo()	boolean
renovar()	void

2.3 Classe CheckIn

Representa o registro de uma visita do aluno à academia.

Atributo	Tipo
dataHora	LocalDateTime
aluno	Aluno
liberado	boolean

Operação	Retorno
registrar()	void
foiLiberado()	boolean

3 DIAGRAMA DE CLASSES

O diagrama de classes a seguir consolida as três classes do domínio, seus atributos, operações e os relacionamentos entre elas, incluindo a especialização AlunoFamilia, detalhada na Seção 6.3.

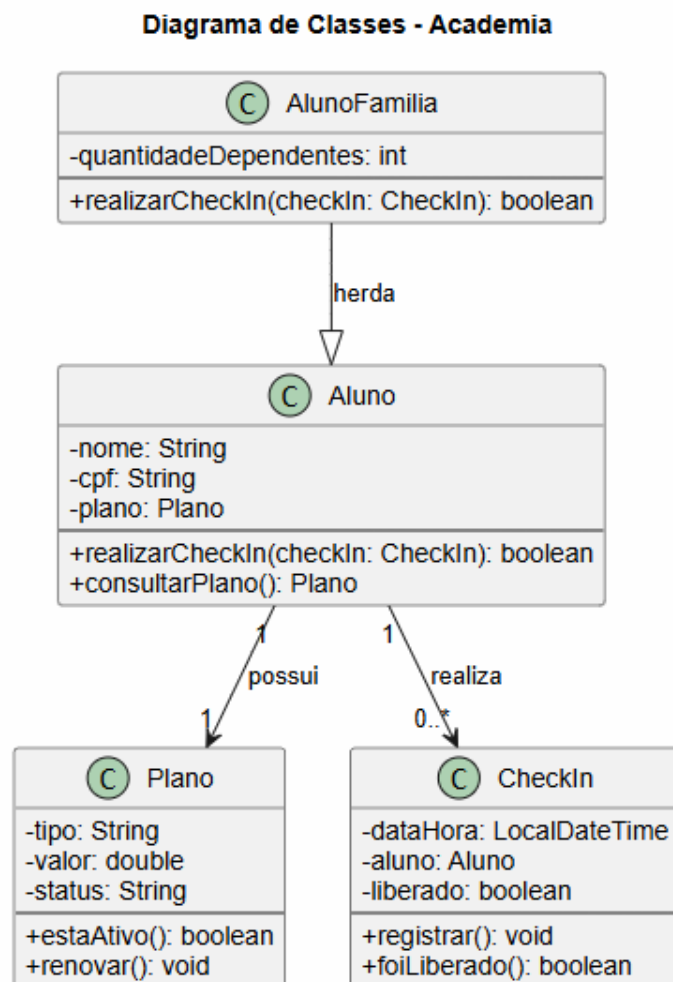


Figura 1 – Diagrama de classes do sistema Academia

Fonte: elaborado pelo grupo (2026).

3.1 Relacionamentos e multiplicidades

- Um Aluno possui um Plano ativo (multiplicidade 1 para 1).
- Um Aluno pode possuir vários registros de CheckIn (multiplicidade 1 para 0..*).

- Cada CheckIn está associado a exatamente um Aluno.

Relacionamentos - Academia

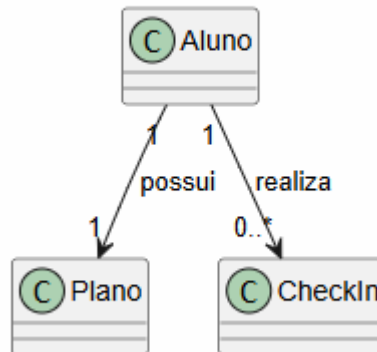


Figura 2 – Relacionamentos e multiplicidades entre Aluno, Plano e CheckIn

Fonte: elaborado pelo grupo (2026).

4 DIAGRAMA DE OBJETOS

Enquanto o diagrama de classes apresenta a estrutura geral do sistema, o diagrama de objetos apresenta um exemplo concreto dos objetos existentes em um determinado momento. A seguir, considera-se um aluno chamado João, com um plano mensal ativo e um check-in já realizado.

Diagrama de Objetos - Academia

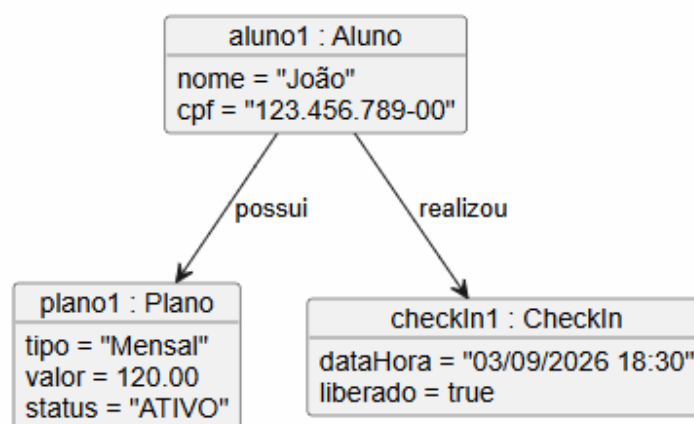


Figura 3 – Diagrama de objetos — instância de Aluno, Plano e CheckIn

Fonte: elaborado pelo grupo (2026).

Nesse exemplo, aluno1 é uma instância da classe Aluno, plano1 é uma instância da classe Plano e checkIn1 é uma instância da classe CheckIn, cada uma com valores reais atribuídos aos seus atributos.

5 MENSAGEM ENTRE OBJETOS

Uma interação típica entre os objetos do sistema ocorre quando um aluno tenta realizar um check-in. A mensagem pode ser representada da seguinte forma:

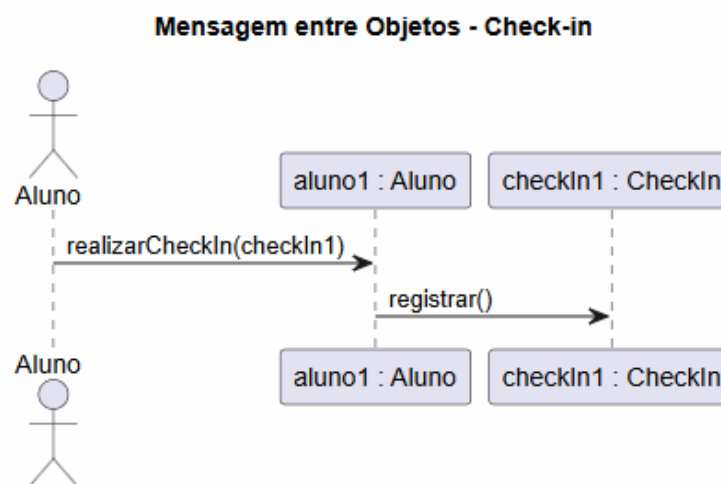


Figura 4 – Mensagem entre os objetos aluno1 e checkIn1

Fonte: elaborado pelo grupo (2026).

Nesse cenário, o objeto aluno1 envia a mensagem realizarCheckIn(checkIn1), que por sua vez aciona a operação registrar() do objeto checkIn1. Durante essa operação, o sistema verifica se o plano associado ao aluno está ativo: caso esteja, o check-in é liberado; caso o plano esteja vencido ou inativo, o check-in é recusado.

5.1 Fluxo de decisão do check-in

O fluxo de decisão executado internamente durante a mensagem realizarCheckIn é detalhado no diagrama de atividades a seguir, que evidencia a comunicação entre os objetos e a regra de negócio aplicada.

Fluxo de Check-in na Academia

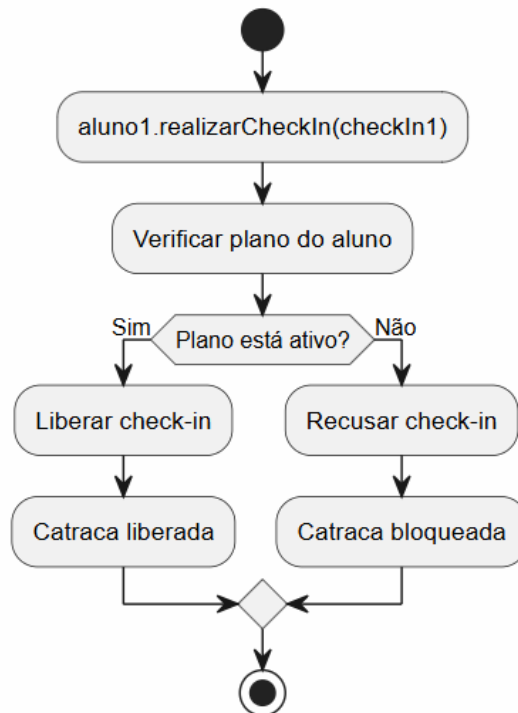


Figura 5 – Fluxo de decisão do check-in na academia

Fonte: elaborado pelo grupo (2026).

6 OS QUATRO PILARES DA ORIENTAÇÃO A OBJETOS

Nesta seção, são apontados, no modelo construído, os pontos em que cada um dos quatro pilares da Orientação a Objetos — Abstração, Encapsulamento, Herança e Polimorfismo — se manifesta.

6.1 Abstração

A abstração está presente na própria criação das classes *Aluno*, *Plano* e *CheckIn*. Cada classe representa uma entidade relevante do domínio da academia e contém somente as informações e os comportamentos necessários para representá-la dentro do sistema. Por exemplo, a classe *Aluno* possui apenas os dados de identificação (nome, CPF) e o plano contratado, além dos comportamentos relacionados ao seu uso dentro da academia — detalhes irrelevantes ao domínio, como o time de futebol do aluno, foram deliberadamente descartados.

6.2 Encapsulamento

O encapsulamento aparece na definição dos atributos como privados, protegendo o estado interno de cada objeto:

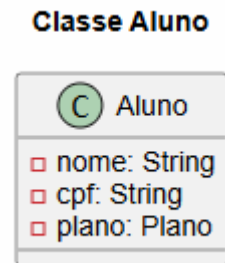


Figura 6 – Atributos privados da classe Aluno

Fonte: elaborado pelo grupo (2026).

Dessa forma, os dados internos do objeto não podem ser modificados diretamente por outras partes do sistema. As alterações devem ocorrer por meio dos comportamentos definidos pelas próprias classes, o que também permite proteger regras do sistema, como impedir que um plano receba um valor inválido ou que um check-in seja registrado com o plano vencido.

6.3 Herança

O projeto também prevê um tipo especial de aluno: o aluno com plano família. Como AlunoFamília é um tipo de Aluno (teste do "é um"), utiliza-se herança para reaproveitar as características já existentes na classe Aluno, em vez de duplicar código.

Herança - Aluno e AlunoFamília

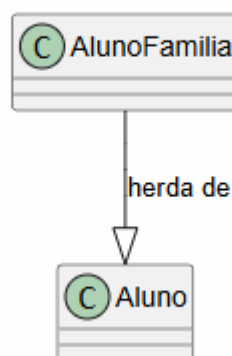


Figura 7 – Herança entre Aluno e AlunoFamília

Fonte: elaborado pelo grupo (2026).

A classe AlunoFamília reutiliza os atributos e as operações de Aluno e acrescenta uma característica específica, a quantidade de dependentes:

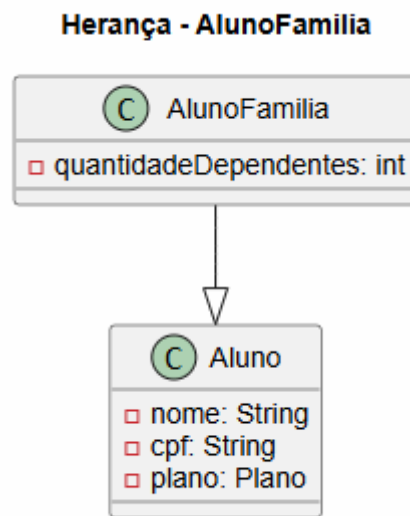


Figura 8 – Atributo específico da classe AlunoFamilia

Fonte: elaborado pelo grupo (2026).

6.4 Polimorfismo

O polimorfismo pode ser utilizado quando diferentes tipos de aluno possuem a mesma operação, mas apresentam comportamentos específicos. Por exemplo, a operação:

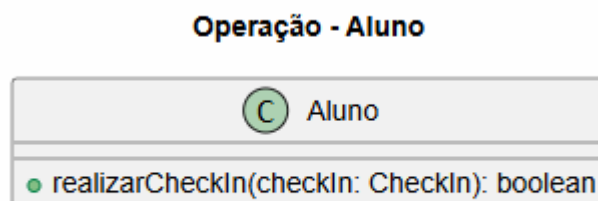


Figura 9 – Operação realizarCheckIn na classe Aluno

Fonte: elaborado pelo grupo (2026).

pode existir na classe Aluno e ser sobrescrita (`@Override`) em uma classe especializada, como AlunoFamilia, caso passem a existir regras específicas para esse tipo de aluno — por exemplo, validações adicionais relacionadas aos dependentes. Assim, o restante do sistema pode trabalhar com uma referência do tipo Aluno, enquanto o comportamento efetivamente executado depende do tipo real do objeto em tempo de execução, evitando a necessidade de blocos condicionais (`if/else` ou `instanceof`) para identificar manualmente cada tipo de aluno.

7 RESUMO DOS CONCEITOS APLICADOS

Pilar	Aplicação no projeto
Abstração	Classes Aluno, Plano e CheckIn representam entidades relevantes do domínio, contendo apenas os dados e comportamentos essenciais.
Encapsulamento	Atributos privados e acesso controlado exclusivamente pelos comportamentos definidos nas próprias classes.
Herança	AlunoFamilia especializa a classe Aluno, reaproveitando atributos e operações e acrescentando a quantidade de dependentes.
Polimorfismo	Diferentes tipos de aluno podem apresentar comportamentos específicos para a mesma operação (realizarCheckIn), sem uso de condicionais por tipo.

8 CONCLUSÃO

O modelo desenvolvido representa uma primeira versão consolidada do sistema de academia, contemplando alunos, planos e registros de frequência. A modelagem permite representar as principais entidades do domínio e seus relacionamentos, além de demonstrar a comunicação entre objetos por meio de mensagens.

Também foram aplicados e identificados no modelo os quatro pilares da Orientação a Objetos: abstração, encapsulamento, herança e polimorfismo, conforme detalhado na Seção 6.

O modelo poderá ser expandido nas próximas etapas do projeto, incluindo regras mais completas para o ciclo de vida dos planos (máquina de estados), formas de pagamento da mensalidade (via interfaces), casos de uso, diagrama de sequência do fluxo de check-in e, por fim, a implementação em Java integrada aos diagramas apresentados, conforme o roteiro de evolução definido para o projeto.